

Machine Learning

Steps to Model a Machine Learning Problem

Machine Learning (ML) is not just about training algorithms. It is a systematic process that starts with understanding a real-world problem and ends with deploying and maintaining a model that delivers value. Each stage is crucial because mistakes made early in the pipeline can significantly affect the final performance.

1. Problem Identification

The first and most important step is to clearly define the problem you want to solve. Before thinking about algorithms, understand the business or real-world objective.

Key Questions

- What problem are we trying to solve?
- Why is solving this problem important?
- Who are the stakeholders or end users?
- What will success look like?

Examples

- Predict customer churn.
- Detect fraudulent transactions.
- Recommend products.
- Classify emails as spam or not spam.

Tasks Involved

- Define objectives.
- Understand business requirements.
- Determine expected outputs.
- Identify constraints (time, cost, privacy, etc.).

2. Determine Whether ML is Needed

Not every problem requires Machine Learning.

Consider ML if:

- Rules are too complex to write manually.
- Large amounts of historical data exist.
- Patterns are difficult for humans to identify.

Avoid ML if:

- A simple rule-based solution works.
- Data is insufficient.
- Interpretability is more important than prediction accuracy.

3. Define the Machine Learning Task

Translate the business problem into an ML problem.

Common ML Tasks

Task	Purpose	Example
Classification	Predict categories	Spam detection
Regression	Predict numerical values	House price prediction
Clustering	Group similar items	Customer segmentation
Recommendation	Suggest relevant items	Netflix recommendations
Anomaly Detection	Detect unusual behavior	Fraud detection
Time Series Forecasting	Predict future values	Sales forecasting

4. Data Collection

Machine Learning models learn from data. The quality and quantity of data largely determine model performance.

Data Sources

- Databases
- APIs

- Sensors
- CSV files
- Web scraping
- Public datasets
- User interactions

Types of Data

- Structured data
- Semi-structured data
- Unstructured data

Goals

- Collect relevant data.
- Ensure enough samples.
- Verify data quality.
- Check for biases.

5. Data Understanding (Exploratory Data Analysis - EDA)

EDA helps understand the characteristics of the dataset before modeling.

Analyze

- Number of records
- Features
- Target variable
- Missing values
- Duplicate records
- Data types
- Outliers
- Class imbalance
- Correlations
- Feature distributions

Common Techniques

- Histograms
- Box plots
- Scatter plots
- Correlation heatmaps
- Pair plots
- Summary statistics

Objectives

- Understand data patterns.
- Detect anomalies.
- Identify relationships.
- Find potential problems.

6. Data Cleaning

Raw data usually contains errors that must be corrected.

Handle

- Missing values
- Duplicate rows
- Incorrect values
- Inconsistent formatting
- Noise
- Outliers

Techniques

- Remove records
- Impute missing values
- Normalize formats
- Correct errors
- Remove irrelevant columns

Garbage In = Garbage Out (GIGO): Poor-quality data leads to poor model performance.

7. Feature Engineering

Feature engineering involves creating meaningful input variables that improve model performance.

Examples

- Extract day/month/year from dates.
- Calculate customer age.
- Create spending ratio.
- Combine related features.
- Generate interaction features.

Feature Selection

Choose only useful features.

Methods include:

- Correlation analysis
- Mutual information
- Recursive Feature Elimination (RFE)
- Feature importance from tree models

Benefits:

- Faster training
- Better accuracy
- Reduced overfitting

8. Data Preprocessing

Prepare the data in a format suitable for ML algorithms.

Common Preprocessing Steps

Encode Categorical Variables

- Label Encoding
- One-Hot Encoding

- Target Encoding

Scale Numerical Features

- Standardization
- Min-Max Scaling
- Robust Scaling

Handle Imbalanced Data

- Oversampling (e.g., SMOTE)
- Undersampling
- Class weighting

Split Data

Typical split:

- Training set: 70–80%
- Test set: 20–30%

9. Model Selection

Choose an appropriate algorithm based on the problem type and data characteristics.

Classification Algorithms

- Logistic Regression
- Decision Trees
- Random Forest
- Support Vector Machine (SVM)
- Naïve Bayes
- K-Nearest Neighbors (KNN)
- Gradient Boosting (XGBoost, LightGBM, CatBoost)
- Neural Networks

Regression Algorithms

- Linear Regression
- Ridge Regression

- Lasso Regression
- Decision Tree Regressor
- Random Forest Regressor
- Gradient Boosting Regressor

Clustering Algorithms

- K-Means
- DBSCAN
- Hierarchical Clustering

10. Model Training

Train the selected algorithm using the training dataset.

During Training

The algorithm:

- Learns patterns.
- Estimates parameters.
- Minimizes a loss function.
- Optimizes predictions.

Important Concepts

- Epochs
- Batch size
- Learning rate
- Optimization algorithms
- Loss functions

11. Model Evaluation

Evaluate how well the model performs on unseen data.

Classification Metrics

- Accuracy

- Precision
- Recall
- F1-score
- ROC-AUC
- Confusion Matrix

Regression Metrics

- Mean Absolute Error (MAE)
- Mean Squared Error (MSE)
- Root Mean Squared Error (RMSE)
- R^2 Score

Validation Techniques

- Train/Test Split
- K-Fold Cross Validation
- Stratified Cross Validation

Goal:

- Ensure the model generalizes well and does not overfit or underfit.

12. Hyperparameter Tuning

Hyperparameters are settings defined before training that influence model performance.

Common Techniques

- Grid Search
- Random Search
- Bayesian Optimization

Examples of Hyperparameters

- Learning rate
- Number of trees
- Maximum tree depth
- Regularization strength

- Batch size

13. Model Interpretation

Understand **why** the model makes certain predictions, especially in high-stakes domains like healthcare or finance.

Interpretation Methods

- Feature Importance
- SHAP (SHapley Additive exPlanations)
- LIME (Local Interpretable Model-agnostic Explanations)
- Partial Dependence Plots

Benefits:

- Build trust.
- Detect bias.
- Explain decisions.
- Meet regulatory requirements.

14. Model Deployment

Deployment makes the trained model available for real-world use.

Deployment Options

- REST APIs
- Mobile/Web applications
- Cloud platforms

Common Deployment Platforms

- AWS
- Microsoft Azure
- Google Cloud Platform (GCP)
- Docker
- Kubernetes

Measuring Errors in Regression (MAE, MSE, RMSE & R² Score)

After training a **regression model**, we need to evaluate how well it predicts continuous numerical values. Evaluation metrics measure the difference between the **actual values** and the **predicted values**, helping us understand the model's accuracy and reliability.

The four most commonly used regression evaluation metrics are:

- **Mean Absolute Error (MAE)**
- **Mean Squared Error (MSE)**
- **Root Mean Squared Error (RMSE)**
- **R² Score (Coefficient of Determination)**

1. Mean Absolute Error (MAE)

MAE (Mean Absolute Error) calculates the average of the **absolute differences** between the actual and predicted values. It measures how far, on average, the predictions are from the true values, regardless of whether the prediction is higher or lower.

Formula

$$MAE = \frac{1}{n} \sum |y - \hat{y}|$$

Characteristics

- Uses the **absolute value** of errors.
- Treats **all errors equally**, regardless of their size.
- Easy to interpret because it is in the **same unit** as the target variable.
- Less sensitive to outliers than MSE or RMSE.
- Based on the **Manhattan Norm (L1 Norm)**, which measures distance by summing absolute differences.

Advantages

- Simple to understand and interpret.
- Robust against extreme errors (outliers).

- Suitable when every error has equal importance.

Disadvantages

- Does not penalize large errors more than small ones.

2. Mean Squared Error (MSE)

MSE (Mean Squared Error) calculates the average of the **squared differences** between the actual and predicted values. By squaring each error, larger mistakes have a much greater impact on the final value.

Formula

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2$$

The formula is annotated with 'Mean' above the fraction, 'Error' above the difference term, and 'Squared' above the exponent.

Characteristics

- Squares each prediction error before averaging.
- Larger errors contribute significantly more than smaller ones.
- Commonly used as a **loss function** during model training.
- Units are the **square of the target variable**, making interpretation less intuitive.

Advantages

- Strongly penalizes large prediction errors.
- Useful when large mistakes are particularly costly.
- Differentiable, making it suitable for optimization algorithms such as Gradient Descent.

Disadvantages

- Highly sensitive to outliers.
- Harder to interpret because the result is in squared units.

3. Root Mean Squared Error (RMSE)

RMSE (Root Mean Squared Error) is the square root of the Mean Squared Error. It measures the average prediction error while giving greater importance to larger errors due to the squaring step in MSE.

Formula

$$RMSE = \sqrt{\sum_{i=1}^n \frac{(\hat{y}_i - y_i)^2}{n}}$$

$\hat{y}_1, \hat{y}_2, \dots, \hat{y}_n$ are predicted values

$y_1, y_2, \dots, y_n$ are observed values

n is the number of observations

Characteristics

- Square root of MSE.
- Gives more weight to larger errors.
- Expressed in the **same unit** as the target variable.
- Based on the **Euclidean Norm (L2 Norm)**, which measures straight-line distance.

Advantages

- Easy to interpret because it uses the original unit.
- Penalizes large errors more heavily than MAE.
- Widely used in regression problems.

Disadvantages

- Sensitive to outliers.
- Can be misleading if the dataset contains extreme values.

4. R² Score (Coefficient of Determination)

The **R² Score** measures how well the regression model explains the variation in the target variable. Instead of measuring prediction error directly, it evaluates the **goodness of fit** of the model.

It compares the model's predictions with a simple baseline model that always predicts the mean of the target values.

Formula

$$R^2 = 1 - \frac{SS_{RES}}{SS_{TOT}} = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

Where,

- SSR is Sum of Squared Regression also known as variation explained by the model
- SST is Total variation in the data also known as sum of squared total
- y_i is the y value for observation i
- $\bar{y}$ is the mean of y value
- $\hat{y}_i$ is predicted value of y for observation i

Interpretation

R ² Score	Meaning
1.0	Perfect prediction
0.9	Excellent model
0.7–0.9	Good model
0.5–0.7	Moderate performance
0	Model performs no better than predicting the mean
< 0	Model performs worse than predicting the mean

Characteristics

- Indicates the proportion of variance explained by the model.
- Values usually range from **0 to 1**, although negative values are possible.

Advantages

- Easy to compare different regression models.
- Measures how well the model explains the target variable.
- Widely reported in regression analysis.

Disadvantages

- Does not indicate whether predictions are unbiased.
- A high R^2 does not always mean the model is good.
- Can increase simply by adding more features, even if they are irrelevant (Adjusted R^2 addresses this issue).

Stratified Split

When building a **Machine Learning (ML) model**, one of the most critical preprocessing steps is **splitting the dataset into training and test sets**. This ensures that the model is trained on one portion of the data and evaluated on completely unseen data, providing a realistic estimate of how well it will perform on new, real-world data.

A poorly performed train-test split can lead to **biased evaluation**, where the model appears to perform well during testing but fails when deployed. Therefore, choosing the right sampling technique is essential.

Why Do We Split the Dataset?

The primary purpose of splitting the dataset is to evaluate the model's **generalization ability**, i.e., its capability to make accurate predictions on unseen data.

A typical split is:

- **Training Set (70–80%)**
 - Used to train the machine learning model.
 - The algorithm learns patterns and relationships from this data.
- **Test Set (20–30%)**
 - Used only after training.
 - Evaluates the model's performance on unseen data.

Data Leakage (Data Snooping Bias)

One of the biggest mistakes in machine learning is **Data Leakage**, also known as **Data Snooping Bias**.

It occurs when information from the **test set unintentionally influences the training process**. As a result, the model learns patterns that it should not have access to, leading to overly optimistic performance metrics.

Causes of Data Snooping Bias

- Exploring the entire dataset before splitting.
- Performing feature engineering on the complete dataset.
- Scaling or normalizing the entire dataset before splitting.
- Selecting features using information from the test set.
- Hyperparameter tuning using the test data.

Consequences

- Inflated accuracy during evaluation.
- Poor real-world performance.
- Model fails to generalize.
- Misleading conclusions about model quality.

Best Practice: Always split the dataset into training and test sets **before** performing any preprocessing, feature selection, or model training.

Random Sampling

The simplest way to split a dataset is through **Random Sampling**, where the data is randomly shuffled and then divided into training and testing sets.

Each observation has an equal probability of being selected for either set.

Limitations of Random Sampling

Although random sampling is simple and effective for many datasets, it may not preserve the distribution of important characteristics.

Suppose a dataset contains:

Income Level	Percentage
Low	50%
Medium	35%
High	15%

A purely random split may produce:

Income Level	Training Set	Test Set
Low	52%	41%
Medium	34%	45%
High	14%	14%

The test set no longer accurately represents the overall population, leading to biased evaluation.

Stratified Sampling

Stratified Sampling is a sampling technique that ensures important subgroups (called **strata**) are represented in the **same proportion** in both the training and test sets.

Instead of randomly selecting samples from the entire dataset, the data is first divided into homogeneous groups (strata), and then samples are drawn proportionally from each group.

This preserves the original distribution of important features.

What is a Strata?

A **strata** (plural: **strata**) is a subgroup of data created based on a particular characteristic.

Examples include:

- Income category
- Gender
- Age group
- Education level
- Disease status
- Customer segment

Each subgroup should maintain the same proportion in both training and test datasets.

Why Use Stratified Sampling?

Stratified sampling ensures that:

- Training and test datasets have similar distributions.
- Important subgroups are not overrepresented or underrepresented.
- Sampling bias is minimized.
- Model evaluation becomes more reliable.
- The model generalizes better to unseen data.

Example: Titanic Dataset

In the **Titanic Dataset**, one of the most important features is **Passenger Class (Pclass)**.

Passengers in different classes (**First Class, Second Class, and Third Class**) had different survival rates. Therefore, it is important that both the training and test datasets contain approximately the same proportion of passengers from each class.

Instead of using pure random sampling, we perform **stratified sampling** based on the **Passenger Class (Pclass)**.

Loading the Dataset

```
import pandas as pd  
# Load the Titanic dataset  
titanic_data = pd.read_csv("titanic.csv")
```

Passenger Class Distribution

The Pclass column already contains three categories:

Passenger Class	Meaning
1	First Class
2	Second Class
3	Third Class

Visualizing the Passenger Class Distribution

```
import matplotlib.pyplot as plt  
titanic_data["Pclass"].value_counts().sort_index().plot.bar(grid=True)  
plt.title("Passenger Class Distribution")  
plt.xlabel("Passenger Class")  
plt.ylabel("Number of Passengers")
```

Purpose of Visualization

The bar chart helps us:

- Understand the distribution of passengers across different classes.
- Check whether each passenger class is well represented.

- Verify that the stratified split preserves these proportions in both the training and test datasets.

Performing Stratified Shuffle Split

```
from sklearn.model_selection import StratifiedShuffleSplit

stratified_splitter = StratifiedShuffleSplit(

    n_splits=1,

    test_size=0.2,

    random_state=42

)

for training_row, testing_row in stratified_splitter.split(titanic_data, titanic_data["Pclass"]):

    stratified_training_data = titanic_data.loc[training_row]

    stratified_testing_data = titanic_data.loc[testing_row]
```

Example 1: Student Exam Scores

Suppose you have a dataset of students with their exam scores.

Student	Score
A	32
B	45
C	67
D	72
E	89
F	95

If you want to perform stratified sampling based on **score ranges**, you cannot directly use the Score column because it contains many different numerical values.

Instead, create score categories (bins).

Step 1: Create the Dataset

```
import pandas as pd

student_data = pd.DataFrame({
    "Student": ["A", "B", "C", "D", "E", "F"],
    "Score": [32, 45, 67, 72, 89, 95]
})
```

Step 2: Create Score Categories

```
student_data["Score_Category"] = pd.cut(student_data["Score"], bins=[0, 40, 60, 80, 100], labels=["Poor", "Average", "Good", "Excellent"])
```

Output

Student	Score	Score_Category
A	32	Poor
B	45	Average
C	67	Good
D	72	Good
E	89	Excellent
F	95	Excellent

Here,

- **0–40** → Poor
- **40–60** → Average
- **60–80** → Good
- **80–100** → Excellent

Now, Score_Category can be used for **Stratified Shuffle Split**.

After Splitting:

```
training_data = training_data.drop("Score_Category", axis=1)
```

```
testing_data = testing_data.drop("Score_Category", axis=1)
```

Why Do We Need pd.cut()?

Suppose score column is:

32, 33, 34, 35, 36, 37, 38, 39, 40, ...

If you stratify directly on Score, almost every value is unique or has very few examples. Stratified sampling cannot maintain meaningful proportions for hundreds of unique values.

After using `pd.cut()`:

Poor

Average

Good

Excellent

There are only **4 categories**, making it easy for Stratified Shuffle Split to preserve the distribution.

Rule of Thumb

Use `pd.cut()` when:

- The feature is **continuous numerical data** (e.g., age, income, salary, marks, height, weight).
- You want to convert it into **meaningful categories (bins)**.
- Those categories will be used for tasks like **stratified sampling**, analysis, or visualization.

If the feature is **already categorical** (e.g., Gender, Pclass, Department, Country), there is **no need** to use `pd.cut()`.